

Saviorganizuojantys neuroniniai tinklai

Pijus Petkevičius

2024 m. rugsėjo 15 d.

Turinys

1	Įvadas	3
1.1	Tikslas	3
1.2	Uždaviniai	3
2	Eksperimentų vykdymas	3
2.1	Duomenys	3
2.2	Programos kodo šaltinis	3
2.3	Parametrai	4
2.3.1	Hiperparametrai	4
3	Programos kodas	4
4	Rezultatai	8
4.1	Kvantavimo paklaida	8
4.2	SOM žemėlapiai	8
5	Išvados	11

1 Įvadas

1.1 Tikslas

Užduoties tikslas - suprogramuoti saviorganizuojančio neuroninio tinklo (žemėlapiu, SOM) mokymo algoritmą, apmokyti jį naudojant irisų duomenis, gauti mokymo aibės klasterizavimo rezultata.

1.2 Uždaviniai

- Normalizuoti irisų duomenų aibės įeitis.
- Rasti SOM programinį kodą, jį pakeisti, kad tiktų irisų duomenims, apskaičiuotų kvantavimo paklaidą.
- Apmokyti SOM irisų duomenimis su skirtingais žemėlapiais: 5x5, 10x10.
- Gauti irisų duomenų klasterizavimo ir kvantavimo paklaidos rezultatus su skirtingais žemėlapio dydžiais: 5x5, 10x10.
- Klasterizavimo rezultatus atvaizduoti lentelėje.

2 Eksperimentų vykdymas

2.1 Duomenys

Šiam darbui naudoti **irisų duomenys**. Šiame rinkinyje yra 150 įrašų, viso yra 3 klasės - 50 įrašų kiekvienoje klasėje.

Šio tyrimo metu visi 150 įrašų buvo panaudoti mokymui, jie taip buvo panaudoti duomenų klasterizavimui.

2.2 Programos kodo šaltinis

Programinis kodas perrašytas pagal šį straipsnį(lengviau išmokti perrašant), jam buvo atlikti šie pakeitimai:

- Norėta turėti galimybę įvesti epochų skaičiaus parametą, duomenų aibės elementai naudojami iš eilės, vienos epochos metu;
- Pridėtas kvantavimo paklaidos skaičiavimas;
- Atlikti pakeitimai, kad SOM atvaizdavimas turėtų irisų klases legendoje;
- Duomenų aibė nebeskaidoma į mokymo ir testavimo aibes.

2.3 Parametrai

2.3.1 Hiperparametrai

- žemėlapių dydis - 5x5 arba 10x10;
- epochų skaičius - 400;
- pradinis mokymo parametras, $\alpha(t) = 0,2$;
- pradinis Manheteno atstumas - 4.

3 Programos kodas

```
1 import numpy as np
2 from math import ceil
3 import matplotlib.pyplot as plt
4 from matplotlib import colors
5
6 numRows = 5          # SOM lentelės eilučių skaičius
7 numCols = 5          # SOM lentelės stulpelių skaičius
8 maxLearningRate = 0.2 # Pradinis mokymo parametras
9 epochs = 400         # Epochų skaičius
10 maxManhattanDistance = 4 # Pradinis Manheteno atstumas
11 fileName = 'iris.data' # Duomenų failo pavadinimas
12 labels = ['Iris-setosa', 'Iris-versicolor', 'Iris-virginica'] #
    ↪ Klasių pavadinimai
13
14 # Irisų duomenų aibėje, duomenų reikšmėms priskiriamas numeris,
15 # duomenys sumaišomi, paskutinis stulpelis (klasės reikšmė)
    ↪ perkeliamas į atskirą masyvą ir išmetamas.
16 def prepareIrisData(inputFile: str) -> (np.ndarray, [int]):
17     outputFile= inputFile.replace('.data', '2.csv')
18     with open(inputFile, 'r') as fileInput:
19         with open(outputFile, 'w') as file:
20             for row in fileInput:
21                 modifiedString = row
22                 for i in range(len(labels)):
23                     modifiedString =
                        ↪ modifiedString.replace(labels[i], str(i))
24                 file.write(modifiedString)
25     inputData = readDataFromFile(outputFile, 0)
26     np.random.shuffle(inputData)
27     output = inputData[:, -1]
28     inputData = np.delete(inputData, -1, axis=1)
29     return inputData, output
30
```

```

31 def readDataFromFile(filename: str, skipRows: int = 1) ->
    ↪ np.ndarray:
32     return np.loadtxt(filename, delimiter=',', skiprows=skipRows)
33
34 def euclideanDistance(x: np.array, y: np.array) -> float:
35     return np.sqrt(np.sum(np.power(x - y, 2)))
36
37 def manhattanDistance(x: np.array, y: np.array) -> float:
38     return np.sum(np.abs(x - y))
39
40 def normalizeData(data: np.ndarray) -> np.ndarray:
41     return (data - np.min(data, axis=0) - np.max(data, axis=0)) /
    ↪ (np.max(data, axis=0) - np.min(data, axis=0))
42
43 # Neuronų nugalėtojo paieška
44 def winningNeuron(data, t, som, numRows, numCols):
45     winner = [0,0]
46     shortestDistance = np.sqrt(data.shape[1]) # Kintamasis
    ↪ inicializuojamas didžiausiu atstumu
47     for row in range(numRows): # skaičiuojamas atstumas tarp
    ↪ duomenų vektorių ir kiekvieno SOM neurono, randamas
    ↪ laimėtojas neuronas
48         for col in range(numCols):
49             distance = euclideanDistance(som[row][col], data[t])
50             if distance < shortestDistance:
51                 shortestDistance = distance
52                 winner = [row,col]
53     return winner
54
55 # Gauname mokymosi parametrą ir didžiausią leistiną manhatano
    ↪ atstumą, kuriuo kaimynai nutolę nuo neurono
    ↪ laimėtojo (priklauso nuo dabartinės epochos)
56 def decay(step, epochs, learningRate, maxManhattanDistance):
57     coefficient = 1.0 - (np.float64(step)/epochs)
58     learningRate = coefficient * learningRate
59     neighbourhoodRange = ceil(coefficient * maxManhattanDistance)
60     return learningRate, neighbourhoodRange
61
62 def SOMTraining(irisData, epochs:int, maxManhattanDistance:int,
    ↪ numRows: int, numCols: int):
63     # Inicializuojama matrica su atsitiktinėmis vektorių reikšmėmis
64     numDimensions = irisData.shape[1]
65     np.random.seed(40)
66     som = np.random.random_sample(size=(numRows, numCols,
    ↪ numDimensions))
67

```

```

68     for epoch in range(epochs):
69         # Nustatomas epochos mokymosi greičio parametras ir
        ↪ didžiausias leistinas manhatano atstumas
70         learningRate, neighbourhoodRange = decay(epoch, epochs,
        ↪ maxLearningRate, maxManhattanDistance)
71         for t in range(irisData.shape[0]):
72             # Randamas neuronas-nugalėtojas duotam vektoriui
73             winner = np.array(winningNeuron(irisData, t, som, numRows,
        ↪ numCols))
74
75             # Atnaujinami neurony svoriai
76             # Naudojame burbuliuko kaimynystės funkciją, kur
        ↪ atnaujinami tik j
77             # neurono-nugalėtojo kaimynus patenkančių vektorių svoriai
78             for row in range(numRows):
79                 for col in range(numCols):
80                     if manhattanDistance(np.array([row,col]),winner) <=
        ↪ neighbourhoodRange:
81                         som[row][col] += learningRate * (irisData[t] -
        ↪ som[row][col])
82
83         return som
84
85     # Kiekvieno langelio spalva parenkama pagal klasę, kuri
        ↪ pasikartoja dažniausiai, jei langelis tuščias, tai
86     # langelio spalva - balta
87     def drawMap(map, numRows, numCols):
88         labelMap = np.zeros(shape=(numRows, numCols), dtype=np.int16)
89         for row in range(numRows):
90             for col in range(numCols):
91                 labelList = map[row][col]
92                 if len(labelList) == 0:
93                     label = 3
94                 else:
95                     label = max(labelList, key=labelList.count)
96                 labelMap[row][col] = label
97
98     cmap = colors.ListedColormap(['tab:blue', 'tab:green',
        ↪ 'tab:red', 'w'])
99     colorMap = plt.imshow(labelMap, cmap=cmap)
100     dataLabels = labels.copy()
101     dataLabels.insert(len(dataLabels), 'No class')
102     cbar = plt.colorbar(colorMap, ticks=range(4))
103     cbar.ax.set_yticklabels(dataLabels)
104
105     plt.show()

```

```

106
107 def getMapAndQuantization(irisData, irisOutput, som):
108     # Apmokytas SOM priskiria kiekvieno žetities vektoriaus klasę
109     ↪ žemėlapiui vietai,
110     # kurioje yra neuronas-nugalėtojas
111
112     map = np.empty(shape=(numRows, numCols), dtype=object)
113     quantizationError = 0
114
115     for row in range(numRows):
116         for col in range(numCols):
117             map[row][col] = []
118
119     for t in range(irisData.shape[0]):
120         winner = winningNeuron(irisData, t, som, numRows, numCols)
121         # Laimėtojo neuronui priskiriama iriso klasės duomenų reikšmė
122         map[winner[0]][winner[1]].append(irisOutput[t])
123         # apskaičiuojamas kvantizacijos klaidos dydis, kuris yra
124         ↪ vidutinis euklido atstumas tarp duomenų vektoriaus ir jo
125         ↪ laimėtojo neurono
126         quantizationError += euclideanDistance(irisData[t],
127         ↪ som[winner[0]][winner[1]])
128     return map, quantizationError/irisData.shape[0]
129
130
131 irisData, irisOutput = prepareIrisData('iris.data')
132 irisData = normalizeData(irisData)
133 irisData = np.array(irisData)
134 irisOutput = np.array(irisOutput)
135 print (irisOutput)
136
137 som = SOMTraining(irisData, epochs, maxManhatanDistance, numRows,
138     ↪ numCols)
139
140 # Apskaičiuojama kvantavimo paklaida
141 map, quantizationError = getMapAndQuantization(irisData,
142     ↪ irisOutput, som)
143 print("Quantization error: ", quantizationError)
144
145 drawMap(map, numRows, numCols)

```

4 Rezultatai

4.1 Kvantavimo paklaida

1 lentelėje 10x10 žemėlapių kvantavimo paklaida 2 kartus mažesnė lyginant su 5x5 žemėlapiu. Galime teigti, kad didesniame, 10x10 žemėlapyje neuronai buvo geriau prisitaikę prie duomenų įėjčių nei mažesniame, 5x5 žemėlapyje.

1 lentelė: Kvantavimo paklaida skirtingo dydžio žemėlapiuose

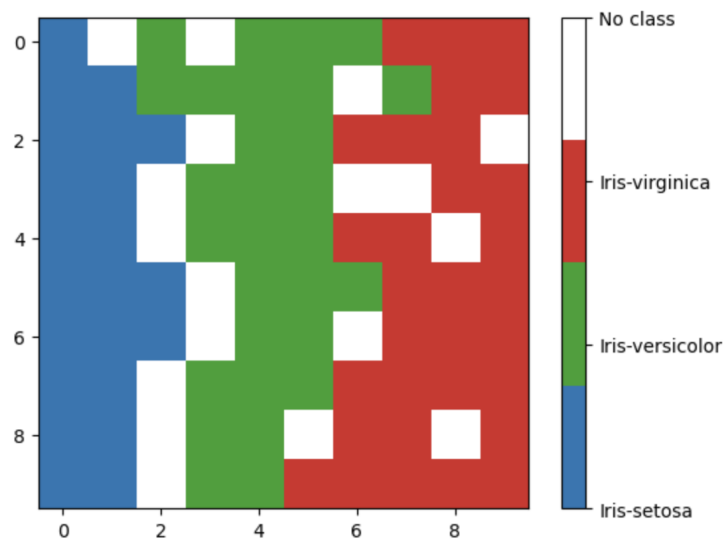
Žemėlapių dydis	Kvantavimo paklaida
5x5	0,1163
10x10	0,0653

4.2 SOM žemėlapiai

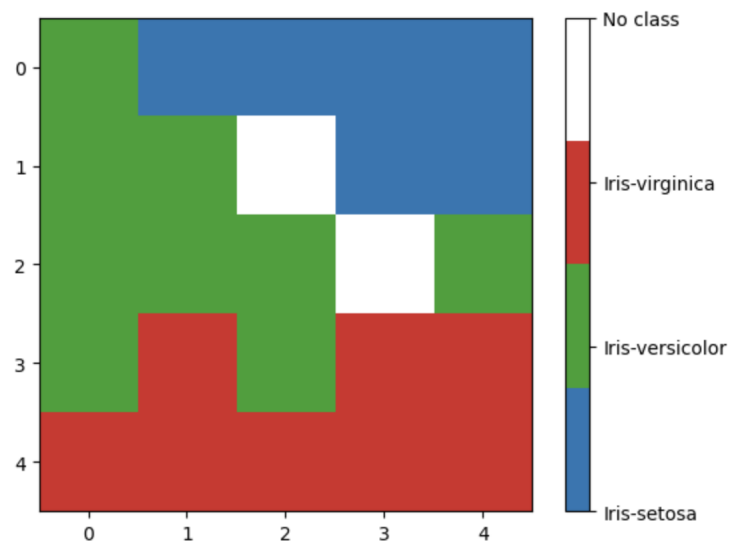
Žemėlapių langeliuose pasirinkta atvaizduoti tą klasę, kurios įeitys buvo dažniausiai priskirtos tam langeliui.

Tiek 1 pav., tiek 2 pav. Iris-versicolor ir Iris-virginica duomenys persikloja kur kas dažniau nei su Iris-setosa. 1 pav. galima aiškiau matyti nei 2 pav., kad kai kurios Iris-versicolor duomenų įeitys panašesnės į Iris-virginica nei į Iris-versicolor.

Tačiau Iris-versicolor klasė artima ir Iris-setosa, nuspelvinti langeliai arti vieni kitų. Labiausiai nutolusios yra Iris-setosa ir Iris-virginica klasės.



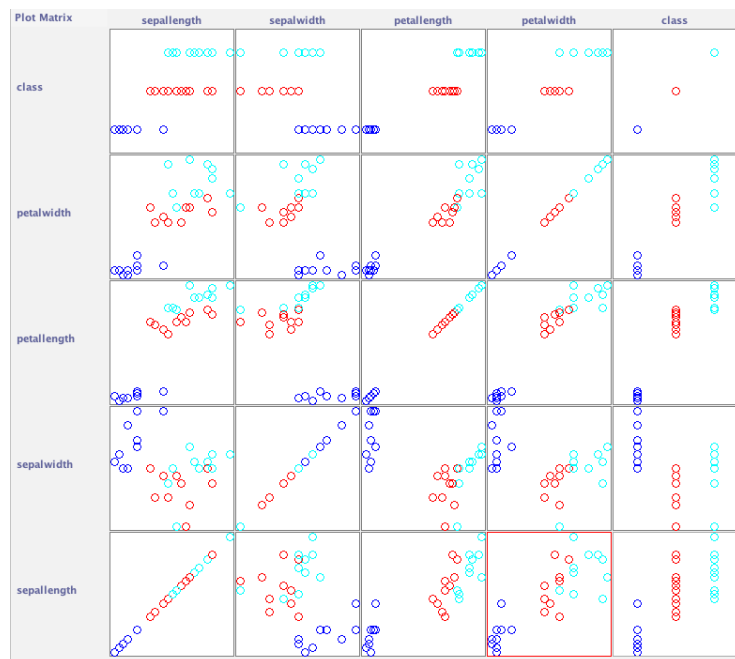
1 pav.: 10x10 žemėlapių klasterizavimo rezultatas



2 pav.: 5x5 žemėlapis klasterizavimo rezultatas

3 pav. pavaizduota mokymo ir testavimo aibės duomenų požymių porų vaizdai. Mėlyna spalva pažymėta Iris-setosa, raudona spalva Iris-versicolor ir žydra spalva Iris-Virginica.

3 pav. galime pastebėti, kad Iris-versicolor ir Iris-virginica požymių porų reikšmės ganėtinai artimos viena kitai, dažnai sutampa, o Iris-setosa klasės požymiai išsiskiria iš kitų dviejų klasių.



3 pav.: Irisų duomenų skirtingų klasių atributų reikšmės

5 Išvados

- 10x 10 žemėlapis geriau išskiria irisų duomenis, lyginant su 5x5 žemėlapiu.
- Iris-virginica ir Iris-versicolor duomenys SOM išsidėsto arti, kartais net persikloja, nes jų duomenų atributai yra ganėtinai artimi.